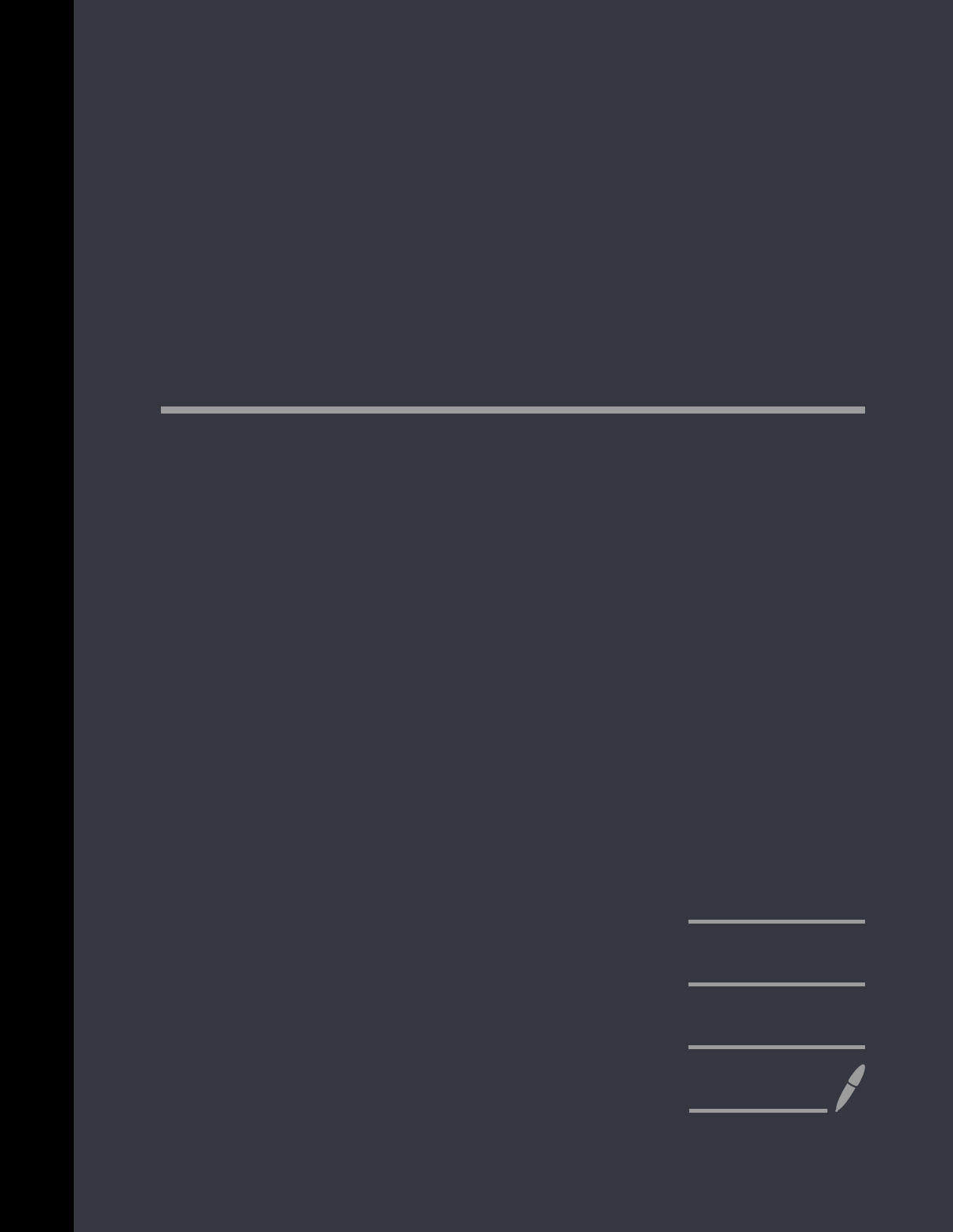


KlausurSS2018ET

Montag, 5. Februar 2024 19:12

Hochschule Aalen

Deckblatt | 19. Juli 2018

Name:	
Vorname:	
Semester:	
Matrikelnummer:	
Semester, in dem Testate bestanden wurden (z.B. SoSe2018):	

Hinweise:

- Bitte legen Sie Ihren Studierendenausweis bereit.
- Notieren Sie auf jedem Blatt Name und Matrikelnummer.
- Bitte lesen Sie die Aufgabenstellungen genau durch.
- Erlaubte Hilfsmittel: keine
- Bitte keinen Rotstift verwenden.
- Die Prüfungsdauer beträgt 90 Minuten.
- 90 Punkte entsprechen 100%.

Viel Erfolg!

Aufgabe	Thema	max. Punkte	erreichte Punkte
1	Rekursion	20	
2	Operatoren	20	
3	Schleifen	20	
4	Funktionen	30	
gesamt		90	

Hochschule Aalen
Technik und Wirtschaft
Beethovenstraße 1
73430 Aalen
www.hs-aalen.de

Prüfung 46101 / 48101 "Programmieren 1"

Studiengang Elektrotechnik, Sommersemester 2018
Prüfer: Dr. Marc Hermann
Juli 2018

Name _____

Matrikelnr _____

Hochschule Aalen

1. Rekursion

Gegeben ist die folgende rekursive Funktion für positive ganze Zahlen n:

$$F(n) = \begin{cases} 1 & \text{für} \quad n \leq 2 \\ (F(n+1) + F(n-1)) & \text{für} \quad \text{ungerade}\, n \\ F(n-3) + 2 & \text{für} \quad \text{gerade}\, n \end{cases}$$

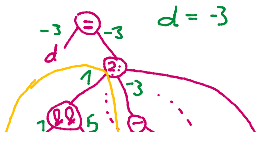
a) Zeichnen Sie einen Aufrufbaum für den Aufruf F(8) und leiten Sie mithilfe des Baums und der Funktion das Ergebnis der Funktion für n=8 her.

b) Zu entwickeln ist nun die rekursive Funktion. Implementieren Sie die Funktion f, indem Sie das folgende Programm ergänzen. Definieren Sie für Zeile 2 den Prototypen (Schnittstelle) der Funktion, ab Zeile 10 kommt dann die Implementierung der Funktion. Die übrigen Programmzeilen müssen Sie nicht nochmals abschreiben. Es reicht, anzugeben, wo Ihre Programmzeilen eingefügt werden.

```
1  #include <stdio.h>
2
3  int main ( ) {
4      int ergebnis, n;
5      scanf("%d", &n);
6      ergebnis = f(n);
7      printf ( "Ergebnis für n=%d: %d \n", n, ergebnis );
8      return 0;
9  }
```

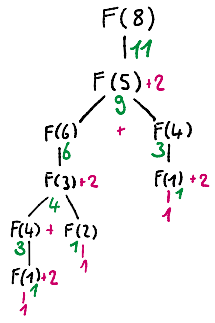
2. Operatoren

Gegeben ist das folgende Programm:



1.

a)

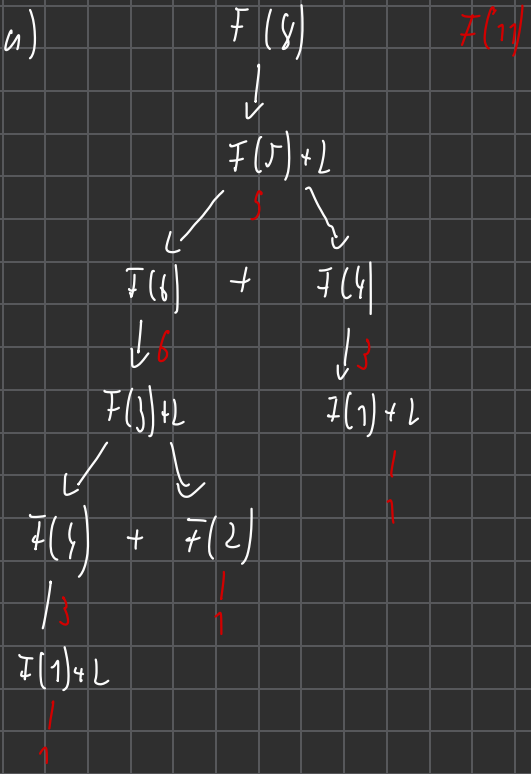


F(8)=11

```
#include <stdio.h>
unsigned int F(unsigned int);
int main(){
    //...
}
unsigned int F(unsigned int n){
    if (n <= 2){
        return 1;
    }
    else if (n % 2 == 1){
        return F(n+1)+F(n-1);
    }
    return F(n-3)+2;
}
```

2.

7a)



b)

```
unsigned int F(unsigned int)

int main () {
   :
unsigned int F(unsigned int n)

  if (n <= 2) {
  return 1; }
  else if (n % 2 == 1) {
  return F(n+1)+(n-1); }
   return F(n-3)+2 }
```

2. Operatoren

Gegeben ist das folgende Programm:

```
1  #include <stdio.h>
2
3  int main ( ) {
4      int a = 4, b = 3, c = 2, d;
5      d = ( (++b == a) && (a = 5) ) ? b - a : c + b + a * c;
6      printf ( "a = %d, b = %d, d = %d \n", a, b++, d );
7      return 0;
8  }
```

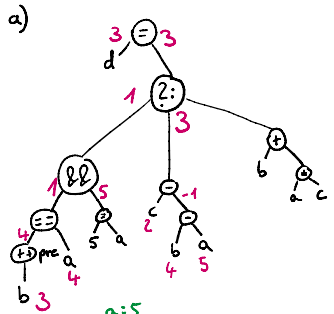
a) Entwickeln Sie den Operatorbaum zum Ausdruck in Zeile 5.

b) Zeigen Sie am Operatorbaum, wie das Ergebnis des Ausdrucks zustande kommt.

c) "a=5, b=4, d=-3

2.

a)



b)

a:5

b:4

c) "a=5, b=4, d=3

"

Name _____

Matrikelnr _____

Hochschule Aalen

3. Schleifen

Gegeben ist das folgende Programm:

```
1   #include <stdio.h>
2   int main ( ) {
3   int i=-1 , j=-1, k=-1;
4   for (i=0; i<=3; i++){
5       j=0;
6       do {
7           printf ( "i=%d j=%d" , i , j++ );
8           for (k = j; k<=i; k++)
9               printf(" k=%d", k);
10          printf ("\n");
11      while (j <= i);
12  }
13   return 0;
14  }
```

a) Entwickeln Sie die exakte Bildschirmausgabe mit kurzer Begründung. Hierfür können Sie z.B. folgendes Tabellen-Schema verwenden:
i j k Zeilennummer Ausgabe

b) Ersetzen Sie die äußere Schleife durch eine kopfgesteuerte Schleife (while), so dass die Ausgabe für alle Schleifendurchläufe gleichbleibt. (Die neue Schleife darf direkt in der Schleife zusätzliche Befehle enthalten, ausgenommen weitere Schleifen)

4. Funktionen

Es ist eine C-Funktion zu entwickeln, welche als Eingabe ein unsortiertes Feld (array) ganzer Zahlen erhält und den Index des ersten Feld-Elements zurückgibt, das nicht durch 2 teilbar ist. Wenn alle Zahlen durch 2 teilbar sind, soll -1 zurückgegeben werden (Fehlerfall).

a) Definieren Sie die Schnittstelle für die Funktion.

b) Entwickeln Sie die Funktion als Struktogramm. Achten Sie auf korrekte Verwendung der Struktogrammsymbole und auf möglichst effiziente Laufzeit. Vermeiden Sie es, im Struktogramm bereits C-Programmcode zu schreiben.

c) Codieren/Implementieren Sie Ihr Struktogramm als C-Funktion.

d) Schreiben Sie ein möglichst kurzes Hauptprogramm, in welchem Sie Ihre Funktion verwenden. Unterscheiden Sie ein "normales" Ergebnis und den Fehlerfall (-1).

3

b)

4. `i=0;`
   `while(i<=3){`
   ...
12. `i++;`
    `}`

i	0	1	2	3	...	groesse-1
array	99	2	99	4		5

b)

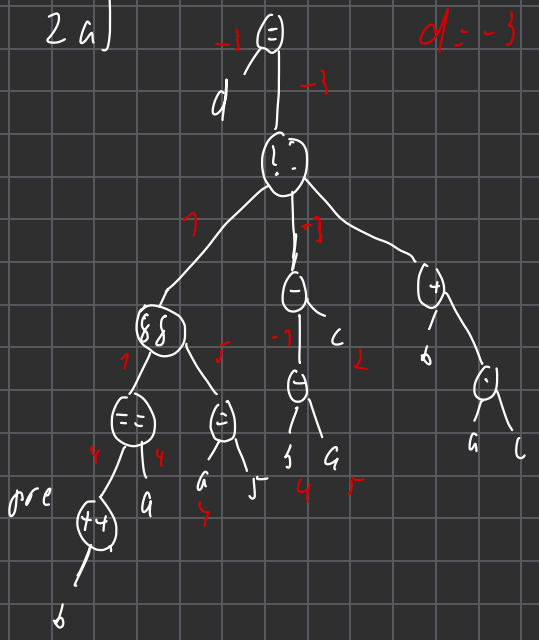
i	j	k	Nr.	Aus
-1	-1	-1	3	
0			4	
	0		5	
			7	"i=0 j=0"
	1	1	8	
			10	"\n"
1	0		5	
			7	"i=1 j=0"
	1	1	8	
			9	"k=1"
	2		8	
			10	"\n"
			7	"i=1 j=1"
	2	2	8	
			10	"\n"
2	0		5	
			7	"i=2 j=0"
	1	1	8	
			9	"k=1"
	2		8	
			9	"k=2"
	3		8	
			10	"\n"
			7	"i=2 j=1"
	2	2	8	
			9	"k=2"
	3		8	
			10	"\n"
			7	"i=2 j=2"
	3	3	8	
			10	"\n"
3	0		5	
			7	"i=3 j=0"
	1	1	9	"k=1"
		2	9	"k=2"
		3	9	"k=3"
		4	10	"\n"
			7	"i=3 j=1"
	2	2	9	"k=2"
		3	9	"k=3"
		4	10	"\n"
			7	"i=3 j=2"
	3	3	9	"k=3"
		4	10	"\n"

4

a) `int Dave(int [], unsigned int);`

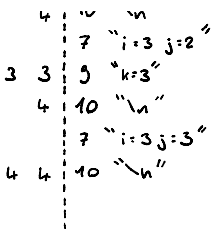
c)
```
int Dave(int [] array, unsigned int groesse)
{
  for(int i=0; i<groesse; i++)
  {
    if(array[i]%2==1)
    {
      return i;
    }
  }
  return -1;
}
int main ()
{
  int a1[] = {2,2,3};
  int erg = Dave(a1,3);
  if (erg == -1)
  {
    printf("Fehler\n");
  } else {
    printf("erfolg, i = %d\n", erg);
  }
}
```

2 a)



d = -3

b) a = 5 b = 4 d = -3


4 "i=3 j=2"
3 3 "k=3"
4 "\n"
"i=3 j=3"
4 4 "\n"